3. GameObject

En Unity, un **GameObject** es la unidad básica de construcción de un juego. Representa cualquier entidad dentro de una escena: personajes, enemigos, balas, elementos del escenario, cámaras, luces, partículas o incluso objetos vacíos que sirven como contenedores lógicos.

Un GameObject por sí mismo no tiene comportamiento. Su funcionalidad se define a través de **componentes**, que se añaden y configuran desde el Inspector o mediante código.

3.1. Estructura de un GameObject

Todos los GameObjects comparten un componente obligatorio:

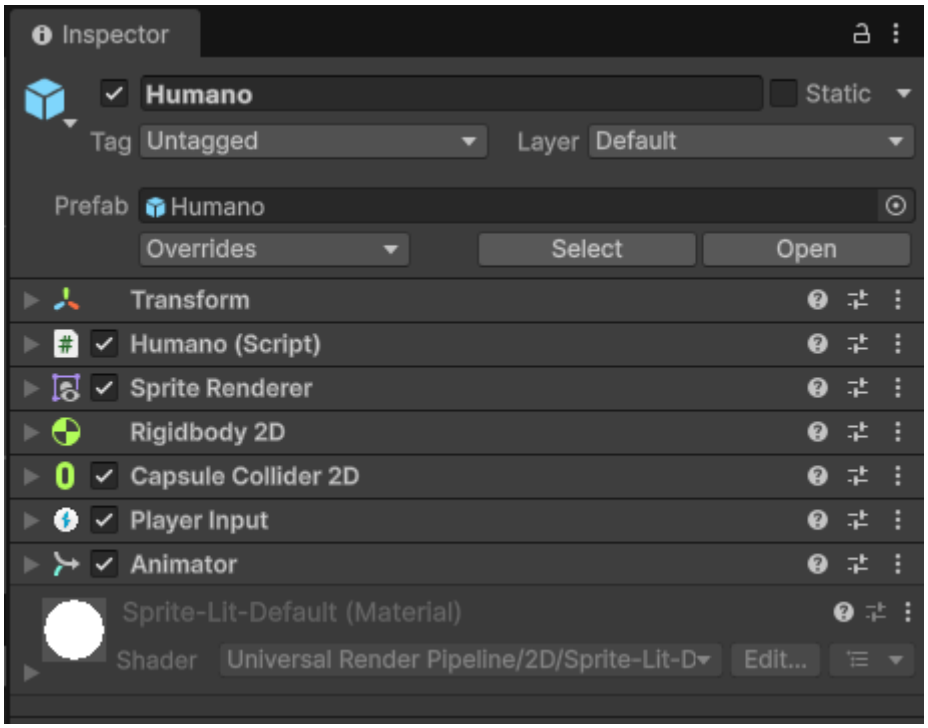
- **Transform** → define posición, rotación y escala.

Además, pueden incluir otros componentes según la función que cumplan:

- Sprite Renderer
- Rigidbody2D
- Collider2D
- AudioSource
- Animator
- Scripts personalizados
- Hijos o subobjetos que complementan su comportamiento

Ejemplo típico de un personaje 2D:

- Humano (GameObject principal)
    - PuntoDisparo (child)
    - Sprite
    - BoxCollider2D
    - Rigidbody2D
    - Animator
    - Script Humano



3.2. Añadir y eliminar componentes

Los componentes pueden agregarse desde:

- **Botón "Add Component"** del Inspector
- Arrastrando un script al Inspector
- Mediante código, por ejemplo:

gameObject.AddComponent<Rigidbody2D>();

También se pueden eliminar desde el menú contextual del Inspector.

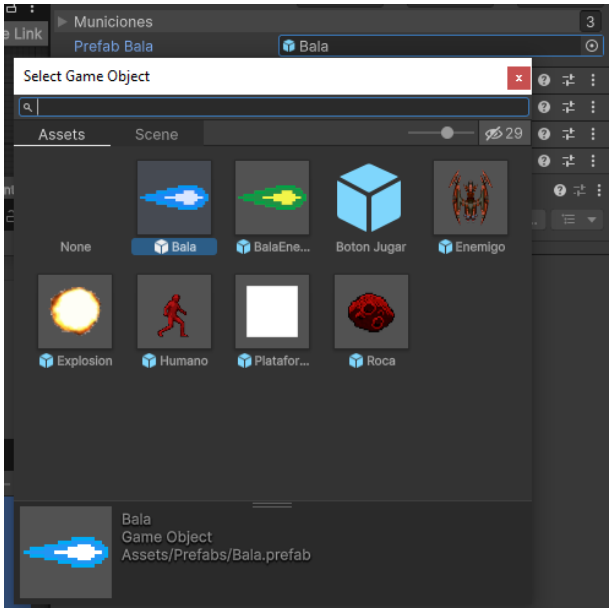
3.3. GameObjects creados mediante Instantiate

Muchos GameObjects no existen en la escena desde el principio, sino que se crean dinámicamente con **Instantiate()** a partir de un Prefab.

Ejemplo tomado del script **Jugador**, donde se crean balas:

```
void CrearBala(Vector3 desplazamiento, float angulo)
{
    Instantiate(prefabBala, transform.position + desplazamiento,
Quaternion.Euler(0, 0, angulo));
}
```

Al instanciar un GameObject, se generan automáticamente todos sus componentes, comportamiento y jerarquía definida en el prefab original.



3.4. GameObjects destruidos mediante Destroy

Los GameObjects pueden ser destruidos en tiempo de juego usando **Destroy()**, lo que elimina el objeto completo y todos sus componentes.

Ejemplo en el script **Enemigo**:

```
Destroy(collision.gameObject);      // Elimina la bala al impactar
```

Y también:

```
Destroy(gameObject);                // Destruye al enemigo cuando su vida llega a cero
```

La destrucción de GameObjects permite gestionar:

- Balas que desaparecen al chocar
- Enemigos eliminados
- Efectos temporales como explosiones

3.5. GameObjects como contenedores de lógica y datos

Los scripts añaden comportamiento a los GameObjects. Por ejemplo, un GameObject puede contener propiedades como:

```
[SerializeField] int vida = 3;

[SerializeField] float velocidad = 5;

[SerializeField] GameObject prefabBalaE;
```

Esto convierte al GameObject en una entidad funcional del juego (enemigo, jugador, etc.), capaz de:

- Detectar colisiones
- Moverse
- Atacar
- Disparar
- Reaccionar a eventos

3.6. GameObjects vacíos

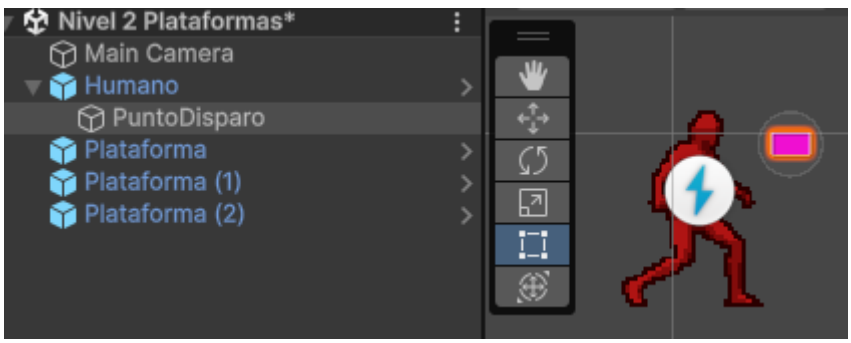
Los GameObjects también pueden ser *vacíos* (solo Transform), y se utilizan frecuentemente como:

- Puntos de referencia
- Contenedores de jerarquía
- Puntos de disparo o aparición
- Controladores lógicos (por ejemplo, un Game Manager)

Ejemplo en el script **Humano**, donde se utiliza un Transform como punto de disparo:

```
[SerializeField] Transform puntoDisparo;
```

Este Transform puede ser un GameObject hijo sin componentes adicionales.



3.7. Activar y desactivar GameObjects

Los GameObjects pueden activarse o desactivarse en tiempo real:

```
gameObject.SetActive(false);
```

Esto permite:

- Mostrar/ocultar elementos del HUD
- Desactivar enemigos temporalmente
- Crear pantallas de menú
- Controlar efectos visuales

3.8. Función del GameObject en el flujo de trabajo

Los GameObjects son el núcleo de Unity. Desde el punto de vista del desarrollo:

- El diseñador coloca GameObjects en la escena.
- El programador escribe scripts que los controlan.
- El artista genera sprites y animaciones que se asignan a ellos.
- El motor gestiona su física, animación y ciclos de vida.

Esta relación convierte al GameObject en el elemento central del motor, uniendo lógica, datos y representación.